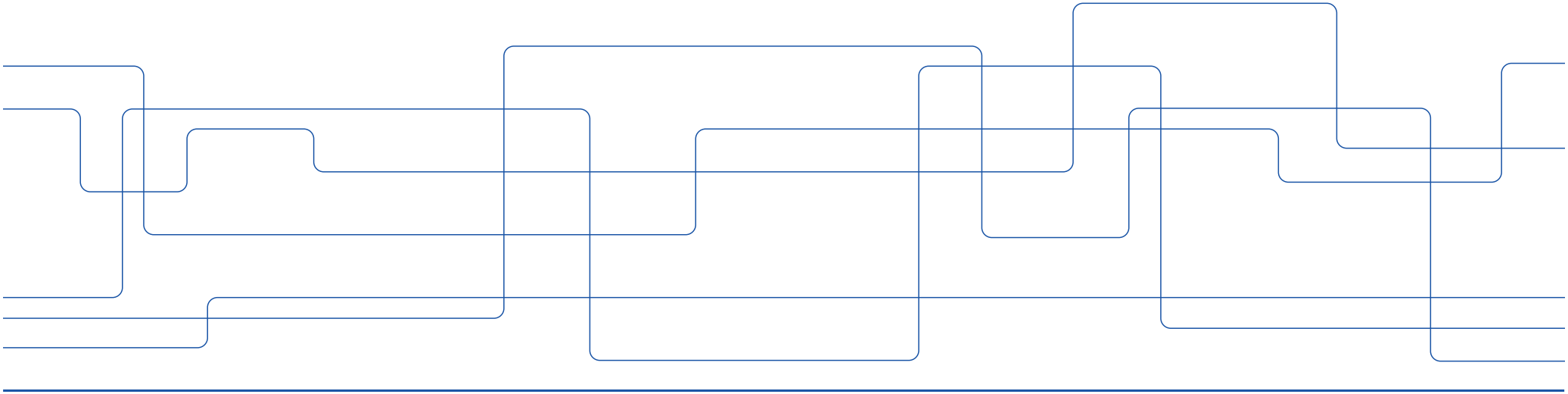


HI1024

Grundkurs i programmering

F6: Matriser (array)!



Innehåll!

King 8

Repetition föreläsning #5

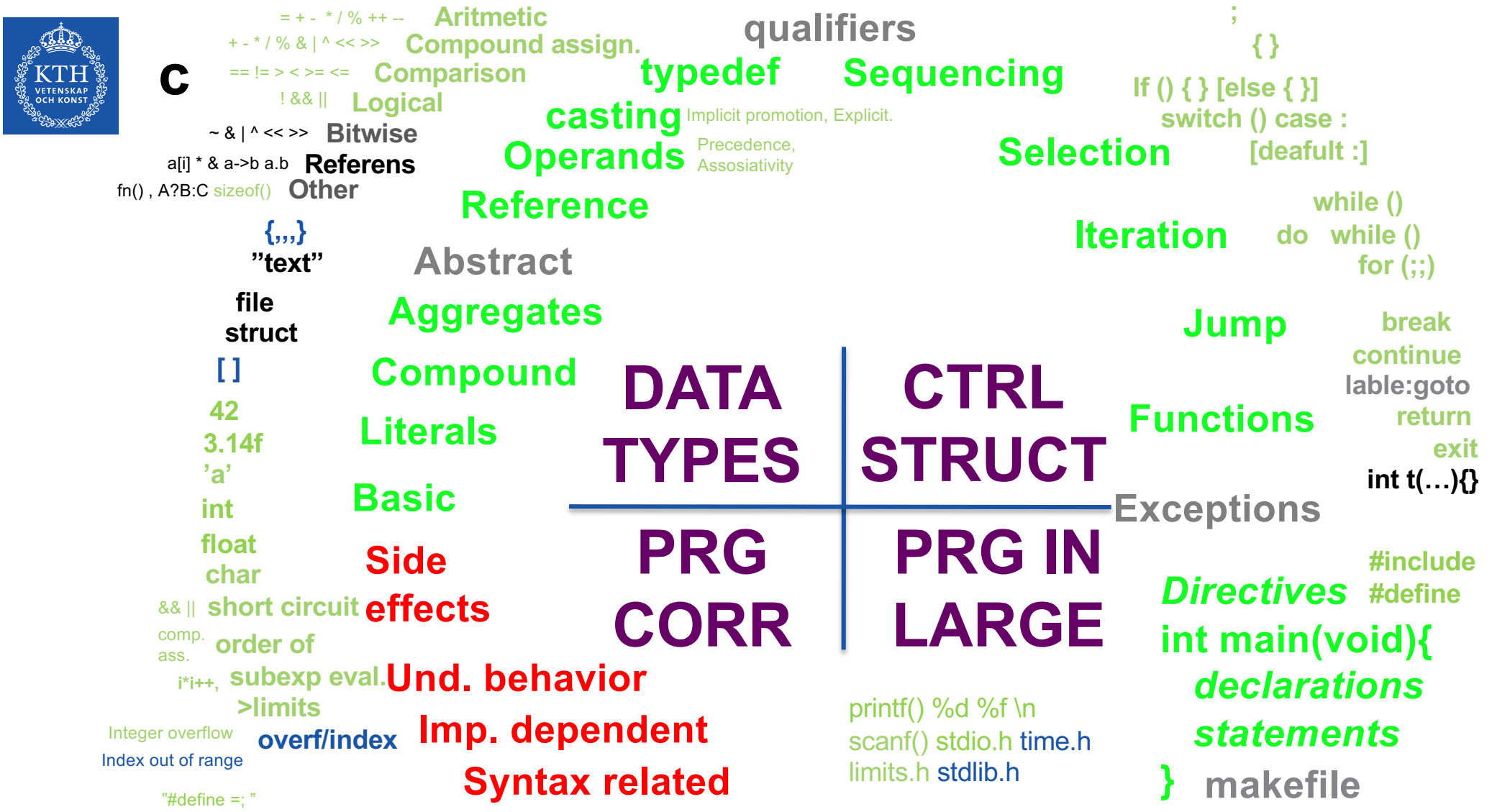
1-dim matriser

Programutvecklingsmodellen

2-dim matriser

Övning




C
= + - * / % ++ --
Aritmetic
+ - * / % & | ^ << >>
Compound assign.
== != > < >= <=
Comparison
! && ||
Logical
~ & | ^ << >>
Bitwise
a[i] * & a->b a.b
Referens
fn() , A?B:C sizeof()
Other
{,,,}
”text”
file
struct
[ ]
42
3.14f
’a’
int
float
char
&& ||
short circuit
comp. ass.
order of
i*i++,
subexp eval.
>limits
Integer overflow
overf/index
Index out of range
”#define =; ”
qualifiers
typedef
Sequencing
casting
Implicit promotion, Explicit.
Operands
Precedence, Assosiativity
Reference
Abstract
Aggregates
Compound
Literals
Basic
Side effects
Und. behavior
Imp. dependent
Syntax related
DATA TYPES
CTRL STRUCT
PRG CORR
PRG IN LARGE
;
{ }
If () { } [else { }]
switch () case :
[deafult :]
Selection
while ()
do while ()
for (;;)
Iteration
Jump
break
continue
lable:goto
return
exit
int t(…){}
Functions
Exceptions
#include
#define
Directives
int main(void){
declarations
statements
}
makefile
printf() %d %f \n
scanf() stdio.h time.h
limits.h stdlib.h
int/int
/*comment*/ or //



limits.h



Typ	minsta värde	största värde	printf,scanf
short int	-32 768	32 767	%hd
unsigned short int	0	65 535	%hu
int	-2 147 483 648	2 147 483 647	%d
unsigned int	0	4 294 967 295	%u
long int	-2 147 483 648	2 147 483 647	%ld
unsigned long int	0	4 294 967 295	%lu

Typ	Minsta positiva värde	Största värde	Presicion	printf, scanf
float	1.17549e-38	3.40282e38	6 siffror	%f, %f
double	2.22507e-308	1.79769e308	15 siffror	%f, %lf

(unsigned long long int 18,446,744,073,709,551,615 är det största exakta talet som finns)

Explicit typkonvertering



(typ) – denna operator konverterar om möjligt talet till höger till datatypen typ.



```
int i = (int) 3.45;
```

Konverteringen sker dock automatiskt vid behov:

```
int i = 3.45;//ger samma resultat

float f = 3 / 2;          (HL int därefter implicit konv. till float)   //f blir 1.0
f = 3.0 / 2;              (HL float)                                    //f blir 1.5

int i1 = 3;
int i2 =2;
f = i1 / i2;              (HL int därefter implicit konv. till float)   //f blir 1.0
f = (float) i1 / i2;      (HL float, tack vare explicit konv. av i1))   //f blir 1.5
```

Datatypen char

```c
/* Characters 0.1 AC */
#include <stdio.h>

int main(void){

    printf("Characters Tester (CT)!\n");
    char ch1 = 'H';
    char ch2 = 'A';
    char ch3 = 'L';

    printf("In what movie could you see\n");
    printf("the %c%c%c9000 computer?\n",ch1,ch2,ch3);

    printf("CT Done.\n");
    return 0;
}
```

```
Characters Tester (CT)!
In what movie could you see
the HAL9000 computer?
CT Done.
```

%c används i printf och scanf för att hantera en (1) bokstav!

```
 PROGRAM
#0000: CODE
#0001: CODE
#0010: CODE
    *
    *
 VART   VÄRDE.       NAMN  TYP
#1000: 0100 1000     ch1   char
#1001: 0100 0001     ch2   char
#1010: 0100 1100     ch3   char
```

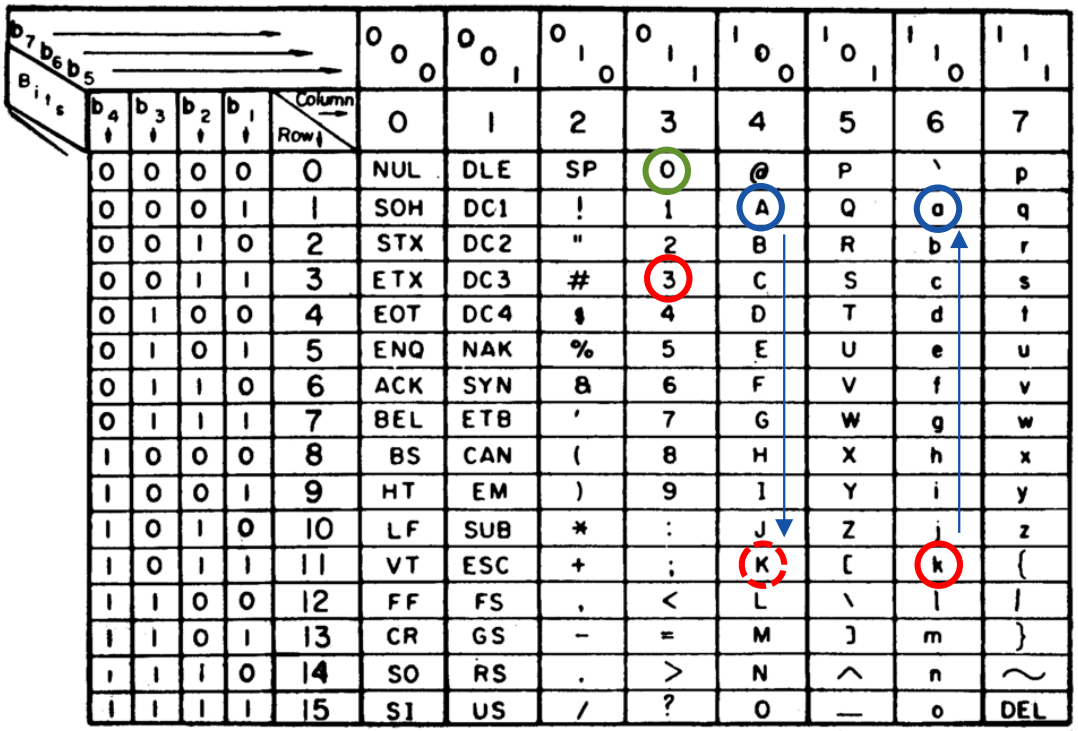
R

USASCII code chart

Bits b7 b6 b5 / Column	0 0 0	0 0 1	0 1 0	0 1 1	1 0 0	1 0 1	1 1 0	1 1 1
b4 b3 b2 b1 / Row 0	0	1	2	3	4	5	6	7
0 0 0 0 0	NUL	DLE	SP	0	@	P	`	p
0 0 0 1 1	SOH	DC1	!	1	A	Q	a	q
0 0 1 0 2	STX	DC2	"	2	B	R	b	r
0 0 1 1 3	ETX	DC3	#	3	C	S	c	s
0 1 0 0 4	EOT	DC4	$	4	D	T	d	t
0 1 0 1 5	ENQ	NAK	%	5	E	U	e	u
0 1 1 0 6	ACK	SYN	&	6	F	V	f	v
0 1 1 1 7	BEL	ETB	'	7	G	W	g	w
1 0 0 0 8	BS	CAN	(	8	H	X	h	x
1 0 0 1 9	HT	EM	)	9	I	Y	i	y
1 0 1 0 10	LF	SUB	*	:	J	Z	j	z
1 0 1 1 11	VT	ESC	+	;	K	[	k	{
1 1 0 0 12	FF	FS	,	<	L	\	l	|
1 1 0 1 13	CR	GS	-	=	M	]	m	}
1 1 1 0 14	SO	RS	.	>	N	^	n	~
1 1 1 1 15	SI	US	/	?	O	_	o	DEL

Eftersom en char är en position i ASCII-alfabetet går det att behandla en char som en int, t.ex är 'A'=65!

Datatypen char



USASCII code chart

b4	b3	b2	b1	Row ↓ / Column →	0 (000)	1 (001)	2 (010)	3 (011)	4 (100)	5 (101)	6 (110)	7 (111)
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	X	h	x
1	0	0	1	9	HT	EM	)	9	I	Y	i	y
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
1	0	1	1	11	VT	ESC	+	;	K	[	k	{
1	1	0	0	12	FF	FS	,	<	L	\	l	|
1	1	0	1	13	CR	GS	-	=	M	]	m	}
1	1	1	0	14	SO	RS	.	>	N	^	n	~
1	1	1	1	15	SI	US	/	?	O	_	o	DEL

R

Otroligt viktigt!

Har du en char variabel som innehåller tecknet ’3’ kan du lätt få fram siffran 3 genom att subtrahera ’0’: 51-48=0!

```
int i='3'-'0';   // Initierar i till (talet) 3!
```

Har du en char variabel som innehåller en liten bokstav kan du lätt få samma stora bokstav genom att subtrahera ’a’ och därefter addera ’A’ (och vise versa)

```
ch=ch-'a'+'A';  // Liten till stor bokstav!
```

Datatypen char

```c
/* Characters 0.4 AC */
#include <stdio.h>

int main(void){

    printf("Characters Tester (CT)!\n");
    char ch;

    do {
        printf("Enter a letter :");
        scanf(" %c",&ch);
        printf("%c [%d]\n",ch,ch);
    } while (ch!='0');

    printf("CT Done.\n");
    return 0;
}
```

```
Characters Tester (CT)!
Enter a letter :b
b [98]
Enter a letter :B
B [66]
Enter a letter :0
0 [48]
CT Done.
```


Mellanslag i en scanf-matchningssträng betyder läs bort 0..n white spaces!

Operatorn sizeof(type)

```c
/* Characters 0.6 AC */
#include <stdio.h>

int main(void){

    printf("Characters Tester (CT)!\n");

    printf("Size of char  : %zu\n", sizeof(char));
    printf("Size of int   : %zu\n", sizeof(int));
    printf("Size of float : %zu\n", sizeof(float));

    printf("CT Done.\n");
    return 0;
}
```

Talar om hur många Byte en datatyp kräver!

```
Characters Tester (CT)!
Size of char  : 1
Size of int   : 4
Size of float : 4
CT Done.
```

Används främst i samband med pekar (senare föreläsning) men finns med i kursen för att påtala ett problem som vi återkommer till!



typedef




```
typedef int Euro;
Euro       wallet;

typedef unsigned int Quantity;
Quantity bults;
```

Wow, vi kan göra egna datatyper!

Otroligt viktigt lite längre fram…

…men bidrar till läsbarhet och att koden lättare kan flyttas mellan olika typer av maskiner!

Observera: När vi skapar en egen datatyp så ska namnet börja med stor bokstav!

Reflektioner övning #5

2=ABC
3=DEF
4=GHI
5=JKL
6=MNO
7=PRS
8=TUV
9=WXY
Otherwize unchanged
All letters uppercase

1-800-COLLECT
1-800-2655328

Verkar omöjligt…
…jag kan inte läsa in alla tecken och sen översätta dom…
…men vart hamnar tecknen när jag skriver in dom?

Yes! I operativsystemets tangentbordsbuffer – så jag kan lungt läsa ett tecken i taget med %c och skriva ut ”rätt” översättning av tecknet!!!

```
Do {
   // Läs in ett (1) tecken (i praktiken alla första gången)
   // switch (ch) {….
   // }
} while (ch!=‘\n’);
```

Reflektioner övning #5

Hum…

…jag ska läsa in bokstäver, minst 1,
men jag vet inte hur många -> do!

…både stora och små bokstäver,
gör jag alla stora så blir det bara
5 tester i stället för 10!

…5 tester är fortfarande många if,
det är nog bättre med en switch!

```
// antal=0
// tecken

// Överskrift

// Fråga efter en mening

// Upprepa…
//    Hämta ett (1) tecken
//    om litet…
//          …gör det stort
//    om…
//          … A E I O U…
//              …antal+1


// …så länge inte radslut

// Visa resultatet!

// Klart!
```

A e i o u
And that's the way it is.
Your sentence contains 6 vowels.

```c
#include <stdio.h>

int main(int argc, char **argv)
{
    int count=0;
    char ch;

    printf("Vowels analyzer 1.0 AC\n");

    printf("Enter a sentence : ");

    do {
        scanf("%c", &ch);
        if (ch>='a')
            ch=ch-'a'+'A';
        switch (ch) {
            case 'A': case 'E':
            case 'I': case 'O':
            case 'U': count++; break;
        }
    } while (ch!='\n');

    printf("Your sentence contains %d vowels\n", count);

    printf("Done!\n");
    return 0;
}
```

Dugga…

```
// Du behöver använda std.h
// Du behöver konstanten pi som är 3,14

// skriv början på main funktionen
// skapa heltalsvariabler i som är 3
// skapa flyttalsvariabler f som är 2,71
// skapa teckenvariabler l som är a
// om i är 3 ska f bli pi
// Skriv ut 7*0..10 tabell

// slutet på main funktionen
```



Dugga…

```
#include <bibliotek>                    // Du behöver använda std.h
#define PI 3.24f                        // Du behöver konstanten pi som är 3,14

int main(void) {                        // skriv början på main funktionen
   int i=1;                             // skapa heltalsvariabler i som är 3
   float f=2.71f;                       // skapa flyttalsvariabler f som är 2,71
   char l='a';                          // skapa teckenvariabler l som är a
   if (i==3) f=PI;                      // om i är 3 ska f bli pi
   for (int j=0; i<11;j++)              // Skriv ut 7*0..10 tabell
      printf("7*%d=%d\n",j,7*j);
}                                       // slutet på main funktionen
```

Skalära, enkla datatyper…



```
PROGRAM
#0000: CODE
   *
 DATA   VÄRDE    NAMN        TYP
#F000:   175     owenTemp    int
#F001:   51.4    beefTemp    float
#F002:   'C'     unitType    char
```

R

Vi tänker oss en 32-bit maskin, och anger innehållet decimalt…

…maskinen har också ett stort minne så vi anger adressen hexadecimalt, tar mindre plats.

…vi kan avbilda verkliga storheter som är heltal, flyttal eller enskilda tecken…

Nytt: matriser…




```
 PROGRAM
#0000: CODE
    *
 DATA   VÄRDE     NAMN          TYP
#F000:
#F001:
#F002:
```


Om vi i ett datorprogram vill hålla koll på antalet ”piller” per dag, hur gör vi då (med det vi kan)?

Nytt: matriser…

```c
/* Characters 0.1 AC */
#include <stdio.h>

int main(void){
    int mo=0,tu=0,we=0,th=0,fr=0,sa=0,su=0;

    printf("Matrix Tester (MT)!\n");

    printf("Monday    : %d\n", mo);
    printf("Tuesday   : %d\n", tu);
    printf("Wednesday : %d\n", we);
    printf("Thursday  : %d\n", th);
    printf("Friday    : %d\n", fr);
    printf("Saturday  : %d\n", sa);
    printf("Sunday    : %d\n", su);

    printf("MT Done.\n");
    return 0;
}
```

```
Matrix Tester (MT)!
Monday    : 0
Tuesday   : 0
Wednesday : 0
Thursday  : 0
Friday    : 0
Saturday  : 0
Sunday    : 0
MT Done.
```

```
 PROGRAM
#0000: CODE
    *
 DATA   VÄRDE    NAMN        TYP
#F000:    0       mo         int
#F001:    0       tu         int
#F002:    0       we         int
#F003:    0       th         int
#F004:    0       fr         int
#F005:    0       sa         int
#F006:    0       su         int
```



…verkligen opraktiskt att skapa 7 olika int variabler, finns det inget bättre sätt?

Nytt: matriser…

Variabel box är en **matris** med WEEKDAY st element!

Alla element har samma typ!

```
int box[WEEKDAYS]={2,1,1,1,0,0,1};
```

Eventuell initiering! (om för få, fyller på 0:or)

Ett enskilt element nås via t.ex box[3], 3:an kallas index…

…och är då som en ”vanlig” variabel av elementtypen…

…det första elementet har indexet 0!

Matriser måste kopieras/jämföras element för element!

```
 PROGRAM
#0000: CODE
    *
 DATA   VÄRDE     NAMN        TYP
#F000:    2       box[0]      int
#F001:    1       box[1]      int
#F002:    1       box[2]      int
#F003:    1       box[3]      int
#F004:    0       box[4]      int
#F005:    0       box[5]      int
#F006:    1       box[6]      int
```



…indexet kan vara vilket ”skumt” uttryck som helst bara resultatet blir ett positivt heltal!

Nytt: matriser…

Wow, nu kan vi mycket enkelt gå igen alla element genom att använda en for-loop (vi vet antalet varv)!

```c
/* Characters 0.2 AC */
#include <stdio.h>
#define WEEKDAYS 7

int main(void){
    int box[WEEKDAYS]={2,1,1,1,0,0,1};

    printf("Matrix Tester (MT)!\n");

    for (int i=0; i<WEEKDAYS; i++)
       printf("Box[%d] : %d\n", i, box[i]);

    printf("MT Done.\n");
    return 0;
}
```

```
Matrix Tester (MT)!
Box[0] : 2
Box[1] : 1
Box[2] : 1
Box[3] : 1
Box[4] : 0
Box[5] : 0
Box[6] : 1
MT Done.
```




```
 PROGRAM
#0000: CODE
    *
 DATA   VÄRDE    NAMN        TYP
#F000:    2      box[0]      int
#F001:    1      box[1]      int
#F002:    1      box[2]      int
#F003:    1      box[3]      int
#F004:    0      box[4]      int
#F005:    0      box[5]      int
#F006:    1      box[6]      int
```



…lättläst och kompakt kod, perfekt, men använd konstanter för storleken: Index out of range!

Nytt: matriser…

”Ideoms”, mycket vanliga konstruktioner

```
for (int i=0; i<SIZE; i++)  /* clears a */
    a[i]=0;

for (int i=0; i<SIZE; i++)  /* reads data */
    scanf("%d",&a[i]);      /* into a */

for (int i=0; i<SIZE; i++)  /* Sums the */
    sum=sum+a[i];           /* elem. of a */

for (int i=0; i<SIZE; i++)  /* Copies a */
    b[i]=a[i];              /* into b */
```

```
 PROGRAM
#0000: CODE
   *
 DATA   VÄRDE    NAMN        TYP
#F000:    0      a[0]         int
#F001:    0      a[1]         int
#F002:    0      a[2]         int
#F003:    0      a[3]         int
#F004:    0      a[4]         int
#F005:    0      a[5]         int
#F006:    0      a[6]         int
```



Observera: I sista exemplet – det går inte att skriva b=a eller b==a!

Nytt: matriser…

Behöver du 1 int skriv: int number = 0;

Behöver du 7 int skriv: int week[7] = {0,1,2,3,4,5,6};

Behöver du 1 float skriv: float pi = 3.1415f;

Behöver du 3 float skriv: float test[3] = {2.2f, 2.3f, 2.1f};

Behöver du 1 char skriv: char unit = 'C';

Behöver du 4 char skriv: char name[4] = {'A','n','n','a'};

```
 PROGRAM
#0000: CODE
   *
 DATA   VÄRDE    NAMN        TYP
#F000:   'A'     name[0]     char
#F001:   'n'     name[1]     char
#F002:   'n'     name[2]     char
#F003:   'a'     name[3]     char
#F004:
#F005:
#F006:
```



Kom ihåg: Första elementet har index 0, för att nå ett element använd [ ], t.ex. name[0] som är 'A'

Nytt: matriser…



```
int a[5];

for (int i=0; i<6; i++)      /* clears a */
    a[i]=0;
```

Fällor!

```
int i=0;

while (i<SIZE)
    b[i]=a[i++];
```

```
 PROGRAM
#0000: CODE
    *
 DATA   VÄRDE    NAMN        TYP
#F000:    ?      a[0]        int
#F001:    ?      a[1]        int
#F002:    ?      a[2]        int
#F003:    ?      a[3]        int
#F004:    ?      a[4]        int
#F005:
#F006:
```



Nytt: matriser…

```
int a[5];

for (int i=0; i<6; i++)      /* clears a */
    a[i]=0;
```

Fällor!

```
int i=0;

while (i<SIZE)
    b[i]=a[i++];
```

```
 PROGRAM
#0000: CODE
    *
 DATA   VÄRDE    NAMN        TYP
#F000:    ?      a[0]        int
#F001:    ?      a[1]        int
#F002:    ?      a[2]        int
#F003:    ?      a[3]        int
#F004:    ?      a[4]        int
#F005:    0      i           int
#F006:
```



Nytt: matriser…

```
int a[5];

for (int i=0; i<6; i++)      /* clears a */
    a[i]=0;
```

Fällor!

```
int i=0;

while (i<SIZE)
    b[i]=a[i++];
```

```
 PROGRAM
#0000: CODE
    *
 DATA   VÄRDE     NAMN        TYP
#F000:    0       a[0]        int
#F001:    ?       a[1]        int
#F002:    ?       a[2]        int
#F003:    ?       a[3]        int
#F004:    ?       a[4]        int
#F005:    0       i           int
#F006:
```



Nytt: matriser…

```
int a[5];

for (int i=0; i<6; i++)      /* clears a */
    a[i]=0;
```

Fällor!

```
int i=0;

while (i<SIZE)
    b[i]=a[i++];
```

```
 PROGRAM
#0000: CODE
    *
 DATA   VÄRDE    NAMN        TYP
#F000:    0      a[0]        int
#F001:    ?      a[1]        int
#F002:    ?      a[2]        int
#F003:    ?      a[3]        int
#F004:    ?      a[4]        int
#F005:    1      i           int
#F006:
```



Nytt: matriser…

```
int a[5];

for (int i=0; i<6; i++)      /* clears a */
    a[i]=0;
```

Fällor!

```
int i=0;

while (i<SIZE)
    b[i]=a[i++];
```

```
 PROGRAM
#0000: CODE
    *
 DATA   VÄRDE    NAMN        TYP
#F000:    0      a[0]        int
#F001:    0      a[1]        int
#F002:    ?      a[2]        int
#F003:    ?      a[3]        int
#F004:    ?      a[4]        int
#F005:    1      i           int
#F006:
```



Snabbspolar fram i=4!

Nytt: matriser…

```
int a[5];

for (int i=0; i<6; i++)      /* clears a */
    a[i]=0;
```

Fällor!

```
int i=0;

while (i<SIZE)
    b[i]=a[i++];
```

```
 PROGRAM
#0000: CODE
    *
 DATA   VÄRDE     NAMN        TYP
#F000:    0       a[0]        int
#F001:    0       a[1]        int
#F002:    0       a[2]        int
#F003:    0       a[3]        int
#F004:    0       a[4]        int
#F005:    4       i           int
#F006:
```



Nytt: matriser…

```
int a[5];

for (int i=0; i<6; i++)      /* clears a */
    a[i]=0;
```

Fällor!

```
int i=0;

while (i<SIZE)
    b[i]=a[i++];
```

```
 PROGRAM
#0000: CODE
    *
 DATA   VÄRDE    NAMN        TYP
#F000:    0      a[0]        int
#F001:    0      a[1]        int
#F002:    0      a[2]        int
#F003:    0      a[3]        int
#F004:    0      a[4]        int
#F005:    5      i           int
#F006:
```



Nytt: matriser…

```
int a[5];

for (int i=0; i<5; i++)   /* clears a */
    a[i]=0;
```

a[5]?!

Fällor!

```
int i=0;

while (i<SIZE)
    b[i]=a[i++];
```

```
PROGRAM
#0000: CODE
   *
DATA   VÄRDE    NAMN        TYP
#F000:   0      a[0]        int
#F001:   0      a[1]        int
#F002:   0      a[2]        int
#F003:   0      a[3]        int
#F004:   0      a[4]        int
#F005:   5      i           int
#F006:
```



Nytt: matriser…

```
int a[5];

for (int i=0; i<6; i++)      /* clears a */
    a[i]=0;
```

Fällor!

```
int i=0;

while (i<SIZE)
    b[i]=a[i++];
```

```
 PROGRAM
#0000: CODE
    *
 DATA   VÄRDE    NAMN        TYP
#F000:    0      a[0]        int
#F001:    0      a[1]        int
#F002:    0      a[2]        int
#F003:    0      a[3]        int
#F004:    0      a[4]        int
#F005:    0      i a[5]      int
#F006:

  #F000 + 5 = #F005 = a[5]
Nollställer i = evighetsloop!
```



C har inga run time tester av index out of bounds, vad som helst kan hända, nästan alltid fatala fel!

Nytt: matriser…

```
int a[5];

for (int i=0; i<6; i++)      /* clears a */
    a[i]=0;
```

Fällor!

```
int i=0;

while (i<SIZE)
    b[i]=a[i++];
```

```
 PROGRAM
#0000: CODE
    *
 DATA   VÄRDE    NAMN        TYP
#F000:    7      a[0]        int
#F001:    3      a[1]        int
#F002:    8      b[0]        int
#F003:    2      b[1]        int
#F004:    ?      c           int
#F005:    0      i           int
#F006:
```



Nytt: matriser…

```
int a[5];

for (int i=0; i<6; i++)      /* clears a */
    a[i]=0;
```

Fällor!

```
int i=0;

while (i<SIZE)
    b[i]=a[i++];
```

```
 PROGRAM
#0000: CODE
    *
 DATA   VÄRDE    NAMN        TYP
#F000:    7      a[0]        int
#F001:    3      a[1]        int
#F002:    8      b[0]        int
#F003:    2      b[1]        int
#F004:    ?      c           int
#F005:    0      i           int
#F006:
```



Nytt: matriser…

```
int a[5];

for (int i=0; i<6; i++)      /* clears a */
    a[i]=0;
```

Fällor!

```
int i=0;

while (i<SIZE)
    b[i]=a[i++];
```

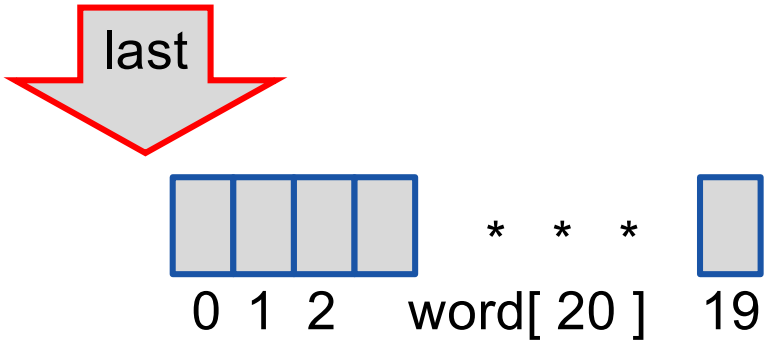
```
 PROGRAM
#0000: CODE
    *
 DATA   VÄRDE    NAMN        TYP
#F000:    7      a[0]        int
#F001:    3      a[1]        int
#F002:    8      b[0]        int
#F003:    2      b[1]        int
#F004:    ?      c           int
#F005:    0      i           int
#F006:
```



Undefined behavior: det är inte säkert att b[0] tilldelas a[0] innan i ökas med 1 (kan bli b[1]=a[0], b[2]=a[1]!

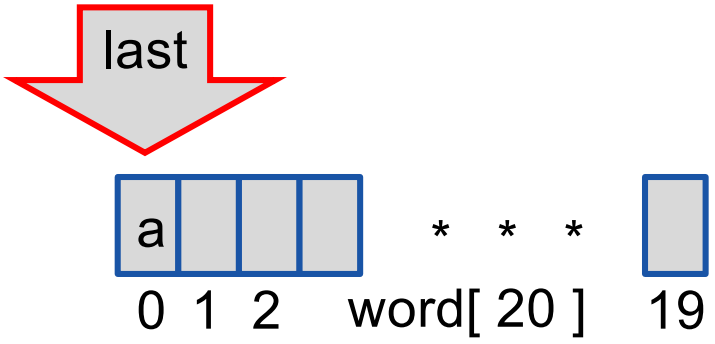


Övning: Palindrom



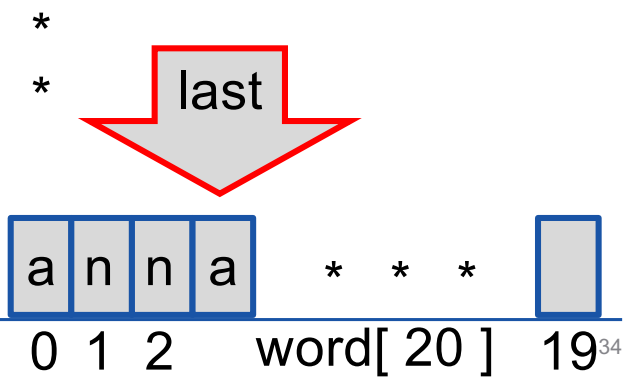
Läs in ett ord…

…och skriv ut det baklänges!



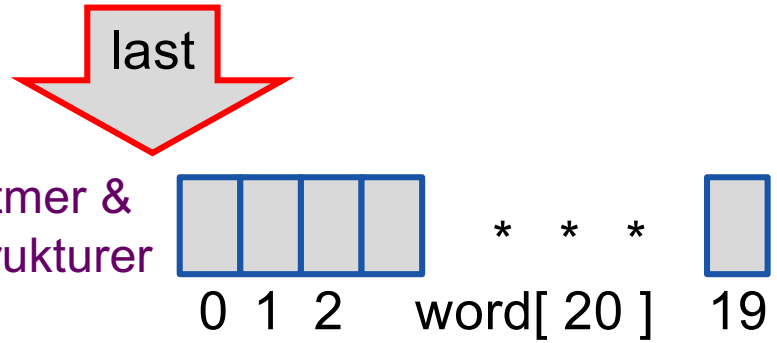
TIPS: Det är smart att först tänka igenom vilken datastruktur som skall användas!

Hum, om jag använder en tecken matris (word) och en heltalsvariabel (last) för vart jag ska lägga in nästa bokstav blir det bra…



Övning: Palindrom

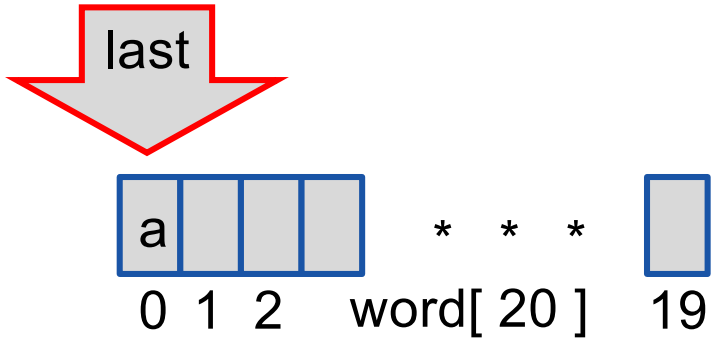
0 Algoritmer & Datastrukturer



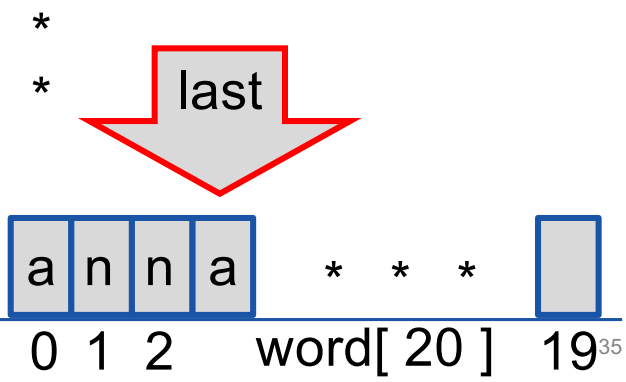
Läs in ett ord…

…och skriv ut det baklänges!

1 **Skriv ner problemet så att ett barn förstår det!**




```
Matrix Tester (MT)!
Enter a word   :naturrutan
it is reversed :naturrutan
MT Done.
```



Övning: Palindrom

```
// WORDMAX 20


// word[WORDMAX]
// sista -1

// Programrubrik

// Instruktion ange ord
// Repetera…
//   läs in bokstav till ord*
// ...tills return

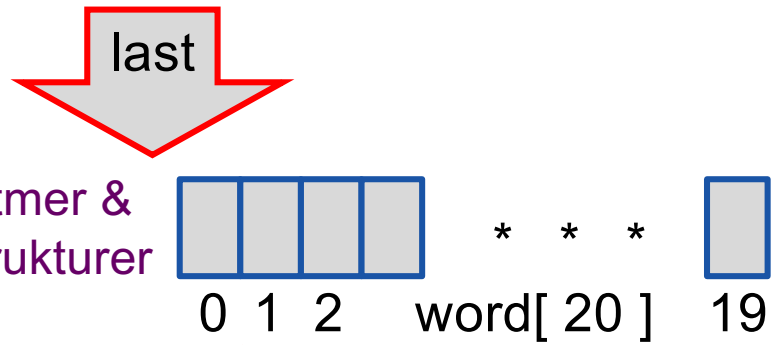
// Visa rubrik
// För alla tecken i ord
//    skriv ut bakifrån
// Gör ny rad

// Klart!
```

Vänta med detta till steg 4:a! använd = {'P','i','a','\n'}; när du skapar variabel word och ge last värdet 3 så ska det fungera!

*öka sista med 1 i smyg!

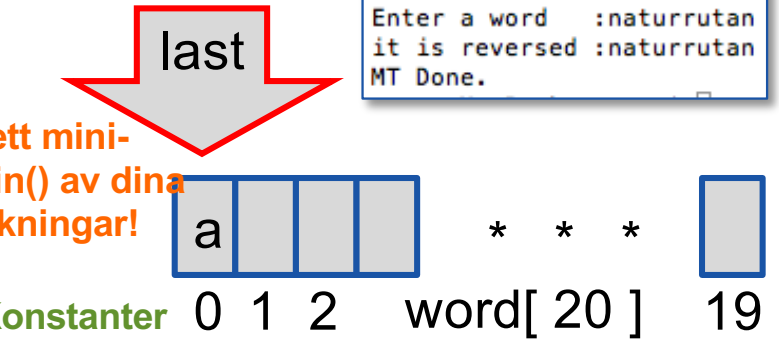
0 Algoritmer & Datastrukturer



1 **Skriv ner problemet så att ett barn förstår det!**

```
Matrix Tester (MT)!
Enter a word   :naturrutan
it is reversed :naturrutan
MT Done.
```

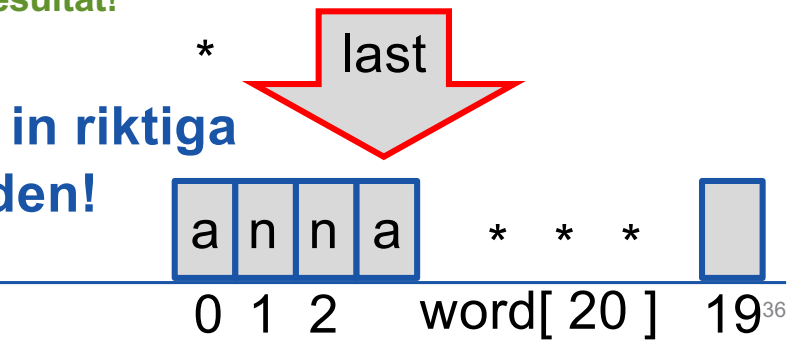
2 **Gör ett mini-malt main() av dina anteckningar!**



3 a..n **Konstanter Variabler Uttryck & resultat!**

```
*
*
```

4 **Läs in riktiga värden!**



(Glöm inte bort att sätta last = -1 när du börjar med steg 4:a)

Övning: Palindrom

```
// WORDMAX 20

// word[WORDMAX]
// sista -1

// Programrubrik

// Instruktion ange ord
// Repetera…
//   läs in bokstav till ord*
// ...tills return

// Visa rubrik
// För alla tecken i ord
//    skriv ut bakifrån
// Gör ny rad

// Klart!

*öka sista med 1 i smyg!
```

```c
/* Characters 0.3 AC */
#include <stdio.h>
#define WORDMAX 20

int main(void){
    char word[WORDMAX];
    int last=-1;

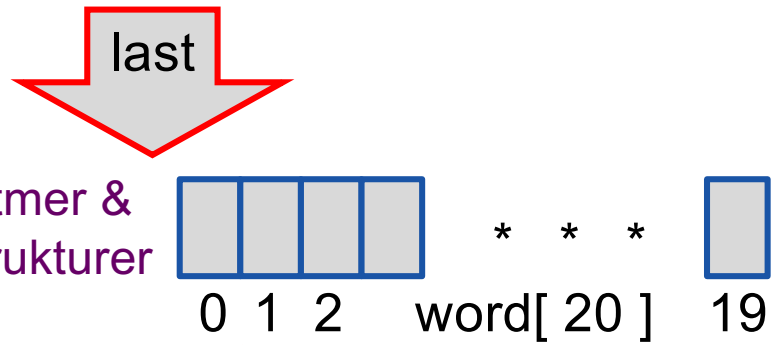
    printf("Matrix Tester (MT)!\n");

    printf("Enter a word   :");
    do
       scanf("%c", &word[++last]);
    while (word[last]!='\n');

    printf("it is reversed :");
    for (int i=last-1; i>=0; i--)
        printf("%c",word[i]);
    printf("\n");

    printf("MT Done.\n");
    return 0;}
```

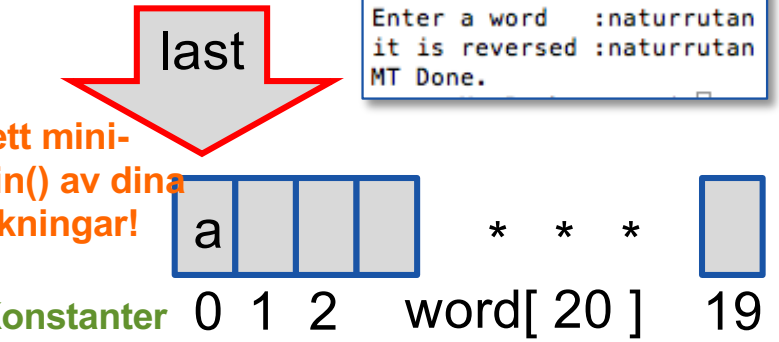
0 Algoritmer & Datastrukturer



1 Skriv ner problemet så att ett barn förstår det!

```
Matrix Tester (MT)!
Enter a word   :naturrutan
it is reversed :naturrutan
MT Done.
```

2 Gör ett mini-malt main() av dina anteckningar!

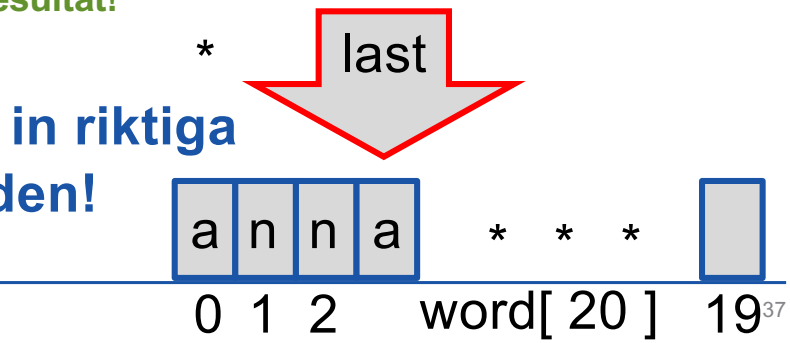


3 Konstanter
Variabler
Uttryck &
resultat!

a..n

```
*
*
```

4 Läs in riktiga värden!



Den här då?



”rader”

```
int w[4][7]={{1,2,3,4,5,6,7},
```

”kolumner”

```
             {1,2,3,4,5,6,7},
             {1,2,3,4,5,6,7},
             {1,2,3,4,5,6,7}};
```

Wow!

Rad 0

Rad 1

```
PROGRAM
#0000: CODE
   *
DATA   VÄRDE    NAMN        TYP
#F000:    1     w[0][0]     int
#F001:    2     w[0][1]     int
#F002:    3     w[0][2]     int
#F003:    4     w[0][3]     int
#F004:    5     w[0][4]     int
#F005:    6     w[0][5]     int
#F006:    7     w[0][6]     int
#F007:    1     w[1][0]     int
#F008:    2     w[1][1]     int
#F009:    3     w[1][2]     int
#F00A:    4     w[1][3]     int
#F00B:    5     w[1][4]     int
#F00C:    6     w[1][5]     int
#F00D:    7     w[1][6]     int
#F00E:    1     w[2][0]     int
```

Det går att ha fler dimensioner också:
char book[PAGES][ROWS][COLUMNS];

Observera hur klamrarna sätts, det ska vara w[rad][kolumn] (inte [rad, kolumn])

Den här då?

```c
/* Characters 0.4 AC */
#include <stdio.h>
#define WEEKDAYS 7
#define DAYSLOTS 4

int main(void){
    int w[DAYSLOTS][WEEKDAYS]= {{1,2,3,4,5,6,7},
                                {1,2,3,4,5,6,7},
                                {1,2,3,4,5,6,7},
                                {1,2,3,4,5,6,7}};

    printf("Matrix Tester (MT)!\n");

    for (int r=0; r<DAYSLOTS; r++) {
      for (int c=0; c<WEEKDAYS; c++)
         printf("%d ",w[r][c]);
      printf("\n");
    }

    printf("MT Done.\n");
    return 0;
}
```




```
Matrix Tester (MT)!
1 2 3 4 5 6 7
1 2 3 4 5 6 7
1 2 3 4 5 6 7
1 2 3 4 5 6 7
MT Done.
```

```
PROGRAM
#0000: CODE
   *
DATA   VÄRDE    NAMN        TYP
#F000:    1     w[0][0]     int
#F001:    2     w[0][1]     int
#F002:    3     w[0][2]     int
#F003:    4     w[0][3]     int
#F004:    5     w[0][4]     int
#F005:    6     w[0][5]     int
#F006:    7     w[0][6]     int
#F007:    1     w[1][0]     int
#F008:    2     w[1][1]     int
#F009:    3     w[1][2]     int
#F00A:    4     w[1][3]     int
#F00B:    5     w[1][4]     int
#F00C:    6     w[1][5]     int
#F00D:    7     w[1][6]     int
#F00E:    1     w[2][0]     int
```



Övning: Frekvenstabell (0..9)!

```
Number Frequence Calculator
Enter a sequence of digits:41271092
 0 :  1
 1 :  2
 2 :  2
 3 :  0
 4 :  1
 5 :  0
 6 :  0
 7 :  1
 8 :  0
 9 :  1
NFC Done - bye!
```

0 Algoritmer & Datastrukturer

Övning: Frekvenstabell (0..9)!

1 **Skriv ner problemet så att ett barn förstår det!**

```
Number Frequence Calculator
Enter a sequence of digits:41271092
 0 :  1
 1 :  2
 2 :  2
 3 :  0
 4 :  1
 5 :  0
 6 :  0
 7 :  1
 8 :  0
 9 :  1
NFC Done - bye!
```

0 Algoritmer & Datastrukturer

Hum… int freq[10]

Bara ental…
…läs in som char…
…kan ta mer än max int!

Minst ett tecken…
…vet inte hur många…
…do loop blir bra!

Övning: Frekvenstabell (0..9)!

```
Number Frequence Calculator
Enter a sequence of digits:41271092
 0 :  1
 1 :  2
 2 :  2
 3 :  0
 4 :  1
 5 :  0
 6 :  0
 7 :  1
 8 :  0
 9 :  1
NFC Done - bye!
```

0 Algoritmer & Datastrukturer

```
// BASE10 10
```

1 **Skriv ner problemet så att ett barn förstår det!**

```
// freq[BASE10]
// char ch

// Välkommen…

// Promt enter digits

// Upprepa…
//    läs in tecken
//    om tal, freq[digit]++
// …så länge inte \n

// Skriv ut freq
```

2 **Gör ett minimalt main() av dina anteckningar!**

Hum… int freq[10]

Bara ental…
…läs in som char…
…kan ta mer än max int!

Minst ett tecken…
…vet inte hur många…
…do loop blir bra!

Övning: Frekvenstabell (0..9)!

```
// BASE10 10

// freq[BASE10]
// char ch

// Välkommen…

// Promt enter digits

// Upprepa…
//    läs in tecken
//    om tal, freq[digit]++
// …så länge inte \n

// Skriv ut freq
```

1 Sk
att

2

```c
#include <stdio.h>
// Konstant 10

int main(int argc, char **argv)
{
    // Frekvenstabell 10 siffror
    // Slaskvariabel för ett tecken

    printf("Number Frequence Calculator\n");

    printf("Enter a sequence of digits:");

    // Upprepa...
        //Läs in tecken
        //om inte \n uppdatera freq[tecken]
    // ...så länge inte \n

    // Skriv ut frekvenstabellen...
    // ...rad för rad!

    printf("NFC Done - bye!\n");
    return 0;
}
```

```
Number Frequence Calculator
Enter a sequence of digits:41271092
 0 :  1
 1 :  2
 2 :  2
 3 :  0
 4 :  1
 5 :  0
 6 :  0
 7 :  1
 8 :  0
 9 :  1
NFC Done - bye!
```

0 Algoritmer & Datastrukturer

Hum… int freq[10]

Bara ental…
…läs in som char…
…kan ta mer än max int!

Minst ett tecken…
…vet inte hur många…
…do loop blir bra!

Ser bra ut…

Övning: Frekvenstabell (0..9)!

```
// BASE10 10

// freq[BASE10]
// char ch

// Välkommen…

// Promt enter digits

// Upprepa…
//    läs in tecken
//    om tal, freq[digit]++
// …så länge inte \n

// Skriv ut freq
```

1 Sk
att

2

3

```
#include <stdio.h>
// Konstant 10

int main(int argc, char **argv)
{
    // Frekvenstabell 10 siffror
    // Slaskvariabel för ett tecken

    printf("Number Frequence Calculator\n");

    printf("Enter a sequence of digits:");

    // Upprepa.
     //Läs in tecken
    // om inte \n uppdatera freq[tecken]
    // ...så länge inte \n

    // Skriv ut frekvenstabellen...
    // ...rad för rad!

    printf("NFC Done - bye!\n");
    return 0;
}
```

Konstanter
Variabler
1..1 Uttryck & resultat!

```
Number Frequence Calculator
Enter a sequence of digits:41271092
 0 :  1
 1 :  2
 2 :  2
 3 :  0
 4 :  1
 5 :  0
 6 :  0
 7 :  1
 8 :  0
 9 :  1
NFC Done - bye!
```

0 Algoritmer & Datastrukturer

Hum… int freq[10]

Bara ental…
…läs in som char…
…kan ta mer än max int!

Minst ett tecken…
…vet inte hur många…
…do loop blir bra!

Övning: Frekvenstabell (0..9)!

```
Number Frequence Calculator
Enter a sequence of digits:41271092
 0 :  1
 1 :  2
 2 :  2
 3 :  0
 4 :  1
 5 :  0
 6 :  0
 7 :  1
 8 :  0
 9 :  1
NFC Done - bye!
```

0 Algoritmer & Datastrukturer

```
// BASE10 10

// freq[BASE10]
// char ch

// Välkommen…

// Promt enter digits

// Upprepa…
//    läs in tecken
//    om tal, freq[digit]++
// …så länge inte \n

// Skriv ut freq
```

1 Sk… att…

2 …

3 Konstanter, Variabler, Uttryck & resultat!

1..2 …

```c
#include <stdio.h>
#define BASE10 10

int main(int argc, char **argv)
{
    int freq[BASE10] = { 0 };
    char ch='2';

    printf("Number Frequence Calculator\n");

    printf("Enter a sequence of digits:");

    // Upprepa...
    //Läs in tecken
    //om inte \n uppdatera freq[tecken]
    // ...så länge inte \n

    // Skriv ut frekvenstabellen...
    // ...rad för rad!

    printf("NFC Done - bye!\n");
    return 0;
}
```

Hum… int freq[10]

Bara ental…
…läs in som char…
…kan ta mer än max int!

Minst ett tecken…
…vet inte hur många…
…do loop blir bra!

Så snart vi har en datastruktur… …se till att du kan skriva ut den!

Övning: Frekvenstabell (0..9)!

```
// BASE10 10

// freq[BASE10]
// char ch

// Välkommen…

// Promt enter digits

// Upprepa…
//    läs in tecken
//    om tal, freq[digit]++
// …så länge inte \n

// Skriv ut freq
```

1 Sk att

2

3 3 3 1 3

Konstanter
Variabler
Uttryck & resultat

```
#include <stdio.h>
#define BASE10 10

int main(int argc, char **argv)
{
    int freq[BASE10] = { 0 };
    char ch='2';

    printf("Number Frequence Calculator\n");

    printf("Enter a sequence of digits:");

    // Upprepa...
    //Läs in tecken
    // om inte \n uppdatera freq[tecken]
    // ...så länge inte \n

    for (int i=0; i<BASE10; i++)
        printf("%2d : %2d\n", i, freq[i]);

    printf("NFC Done - bye!\n");
    return 0;
}
```

```
Number Frequence Calculator
Enter a sequence of digits:41271092
 0 :  1
 1 :  2
 2 :  2
 3 :  0
 4 :  1
 5 :  0
 6 :  0
 7 :  1
 8 :  0
 9 :  1
NFC Done - bye!
```

0 Algoritmer & Datastrukturer

Hum… int freq[10]

Bara ental…
…läs in som char…
…kan ta mer än max int!

Minst ett tecken…
…vet inte hur många…
…do loop blir bra!

…återstår själva inmatningsloopen…

Övning: Frekvenstabell (0..9)!

```
// BASE10 10

// freq[BASE10]
// char ch

// Välkommen…

// Promt enter digits

// Upprepa…
//    läs in tecken
//    om tal, freq[digit]++
// …så länge inte \n

// Skriv ut freq
```

1 Sk
att

2

3

4 Läs in riktiga värden!

```
#include <stdio.h>
#define BASE10 10

int main(int argc, char **argv)
{
    int freq[BASE10] = { 0 };
    char ch='2';

    printf("Number Frequence Calculator\n");

    printf("Enter a sequence of digits:");

    do {
        //Läs in tecken
        if (ch!='\n') freq[ch-'0']++;
    } while (ch!='2');

    for (int i=0; i<BASE10; i++)
        printf(" %2d : %2d\n", i, freq[i]);

    printf("NFC Done - bye!\n");
    return 0;
}
```

```
Number Frequence Calculator
Enter a sequence of digits:41271092
 0 :  1
 1 :  2
 2 :  2
 3 :  0
 4 :  1
 5 :  0
 6 :  0
 7 :  1
 8 :  0
 9 :  1
NFC Done - bye!
```

0 Algoritmer & Datastrukturer

Hum… int freq[10]

Bara ental…
…läs in som char…
…kan ta mer än max int!

Minst ett tecken…
…vet inte hur många…
…do loop blir bra!

Övning: Frekvenstabell (0..9)!

```
// BASE10 10

// freq[BASE10]
// char ch

// Välkommen…

// Promt enter digits

// Upprepa…
//    läs in tecken
//    om tal, freq[digit]++
// …så länge inte \n

// Skriv ut freq
```

1 Sk
att

2

3

4

```c
#include <stdio.h>
#define BASE10 10

int main(int argc, char **argv)
{
    int freq[BASE10] = { 0 };
    char ch='2';

    printf("Number Frequence Calculator\n");

    printf("Enter a sequence of digits:");

    do {
        scanf("%c", &ch);
        if (ch!='\n') freq[ch-'0']++;
    } while (ch!='\n');

    for (int i=0; i<BASE10; i++)
        printf("%2d : %2d\n", i, freq[i]);

    printf("NFC Done - bye!\n");
    return 0;
}
```

```
Number Frequence Calculator
Enter a sequence of digits:41271092
 0 :  1
 1 :  2
 2 :  2
 3 :  0
 4 :  1
 5 :  0
 6 :  0
 7 :  1
 8 :  0
 9 :  1
NFC Done - bye!
```

0 Algoritmer & Datastrukturer

Hum… int freq[10]

Bara ental…
…läs in som char…
…kan ta mer än max int!

Minst ett tecken…
…vet inte hur många…
…do loop blir bra!

Nytt: Slumptal!



Det finns ingen dator som är bra på slump…

…men det går ofta på något sätt att efterlikna slump…

…i c-biblioteket **stdlib.h** finns funktionen **srand()** som med hjälp av ett slumpfrö initierar en slumptalsgenerator. Slumptalen, som är mellan 0 och maxint, hämtas med funktionen **rand()**. Slumpfröet brukar ofta vara datorns klocka och tiden hämtas via **time.h** funktionen **time()**. Satserna blir: **srand((unsigned)time(NULL)); och rnd = rand();**

(Det går att köra utan att srand() anropats, men då blir slumpserien exakt lika varje gång programmet körs!)

Nytt: Slumptal!



```c
/* Characters 0.6 AC */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 6
#define HAND 5

int main(void){

    printf("Matrix Tester (MT)!\n");

    srand((unsigned) time(NULL));    /* Init random generator */

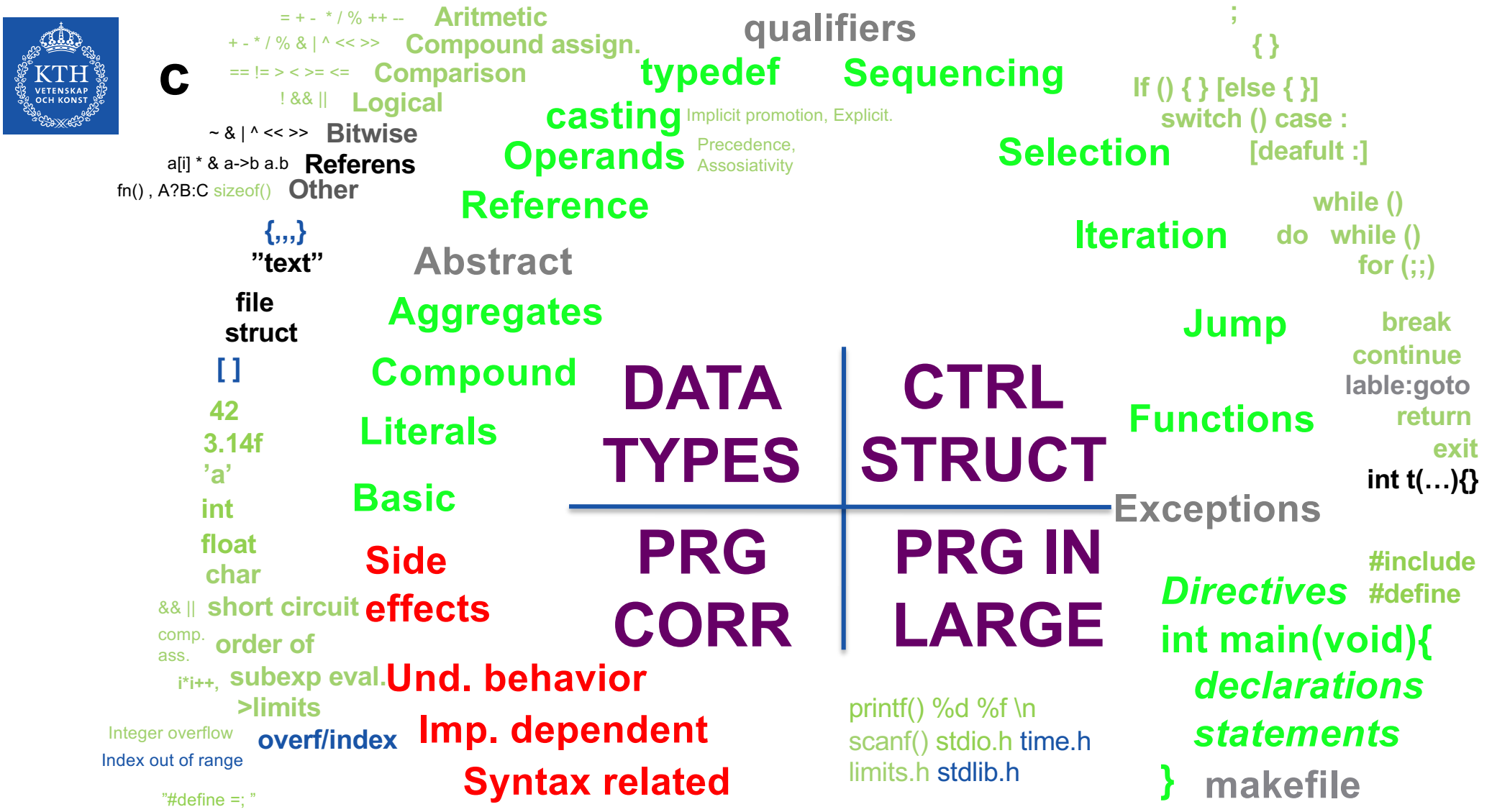
    for (int i=0; i<HAND; i++)
       printf("Dice %d : %d\n", i+1, (rand() % SIZE) + 1);

    printf("MT Done.\n");
    return 0;
}
```

```
Matrix Tester (MT)!
Dice 1 : 6
Dice 2 : 2
Dice 3 : 1
Dice 4 : 2
Dice 5 : 4
MT Done.
vcxxs-macbook:~ vcxx
Matrix Tester (MT)!
Dice 1 : 2
Dice 2 : 3
Dice 3 : 5
Dice 4 : 2
Dice 5 : 2
MT Done.
vcxxs-macbook:~ vcxx
Matrix Tester (MT)!
Dice 1 : 4
Dice 2 : 3
Dice 3 : 1
Dice 4 : 1
Dice 5 : 2
MT Done.
```

Yatzy 0.1

rand() ger ett tal mellan 0 och maxint, men ta modulus (%) 6 så får du ett tal mellan 0 och 5, perfekt!


C
= + - * / % ++ --
Aritmetic
+ - * / % & | ^ << >>
Compound assign.
== != > < >= <=
Comparison
! && ||
Logical
~ & | ^ << >>
Bitwise
a[i] * & a->b a.b
Referens
fn() , A?B:C sizeof()
Other
{,,,}
”text”
file
struct
[ ]
42
3.14f
’a’
int
float
char
Basic
Literals
Compound
Abstract
Aggregates
Reference
qualifiers
typedef
Sequencing
casting
Implicit promotion, Explicit.
Operands
Precedence, Assosiativity
;
{ }
If () { } [else { }]
switch () case :
[deafult :]
Selection
while ()
do while ()
for (;;)
Iteration
Jump
break
continue
lable:goto
return
exit
int t(…){}
Functions
Exceptions
DATA TYPES
CTRL STRUCT
PRG CORR
PRG IN LARGE
#include
#define
Directives
int main(void){
declarations
statements
}
makefile
&& || short circuit
comp. ass.
order of
i*i++,
subexp eval.
Side effects
Und. behavior
Imp. dependent
Syntax related
>limits
Integer overflow
Index out of range
overf/index
”#define =; ”
printf() %d %f \n
scanf() stdio.h time.h
limits.h stdlib.h
int/int
/*comment*/ or //

Övn #6: Kapitel 8:P6 & P9 (P2)

A-4
B-8
E-3
I-1
O-0
S-5
Else unchanged
All letters uppercase
Till matris!!!

Hey dude, C is…
H3Y DUD3, C 15…

```
Tic Tac Toe!
Enter rowcol
or 00 to end
+++
+++
+++
Where?22
+++
+X+
+++
Where?11
O++
+X+
+++
Where?12
OX+
+X+
+++
Where?32
OX+
+X+
+O+
Where?13
OXX
+X+
+O+
Where?21
OXX
OX+
+O+
Where?31
OXX
OX+
XO+
Where?00
Done, bye!
```

A . . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .
. . . . . .

10*10 A..Z

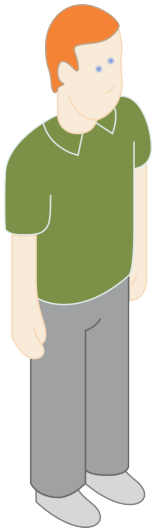
1 Skriv ner problemet så att ett barn förstår det!

2 Gör ett minimalt main() av dina anteckningar!

3 Konstanter
Variabler med
Värde Uttryck
a..n & resultat!

4 Läs in riktiga värden!

Historik



• 150715 0.1	AC	Skapad.
• 150716 0.2	AC	Skalära datatyper
• 150717 0.5	AC	1-dim
• 150718 0.9	AC	2-dim
• 150719 1.0	AC	Slump
• 170905 1.1	AC	Bättre övningar, slump till F7.
• 180914 1.2	AC	Extra övning frekvenstabell.
• 190801 2.0	AC	Omarbetad från grunden, samt anpassad till KTH:s nya profil.